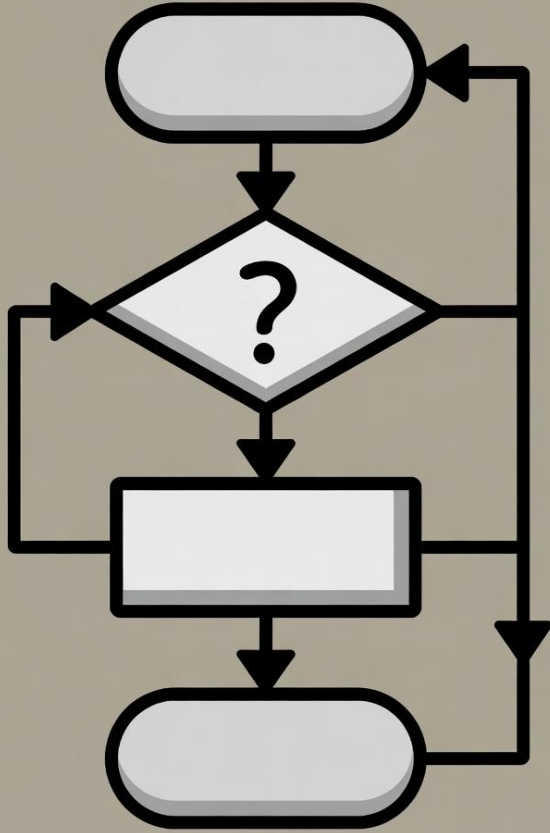




알고리즘 (Algorithm)

AI 신약

신현길

Download Tranches

1,916 (Non-Empty) Tranches Selected (872,843,544 Substances)

AAAA AAAB AAAC AAAD AABA AABB AABC AABD AACA AACB AACC AACD AAEA
AAEB AAEC AAED BAAA BAAB BAAC BAAD BABA BABB BABC BABD BACA
BACB BACC BACD BAEB BAEB BAEC BAED CAAA CAAB CAAC CAAD CABA
CABB CABD CABD CACA CACB CACC CACD CAEA CAEB CAEC CAED DAAA
DAAB DAAC DAAD DABA DABB DABC DABD DACA DADB DACC DADC DAEA
DAEB DAEC DAED EAAA EAAB EAAC EAAD EABA EABB EABC EABD EACA

Only If Modified Since

연도-월-일

Tab-Delimited (*.txt)

Raw URLs

Download

LogP (up to)

Totals, by Weight

Totals, by
LogP

AAAA - Excel

파일 홈 삽입 페이지 레이아웃 수식 데이터 검토 보기 도움말

잘라내기
붙여넣기
복사
서식 복사

맑은 고딕 11 가 가
가 가 간
글꼴 크기
색상
배경색

클립보드 글꼴 맞춤 표시 형식 스타일

자동 줄 바꿈
병합하고 가운데 맞춤
일반
조각
표
표준
나쁨
보통
경고문
계산

	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S
1	smiles	zinc_id	inchkey	mw	logp	reactive	purchas	abtranche_n	features										
2	CO[C@H]	4371221	ZBDGHWf	164.157	-1.928	0	50	AAAA											
3	C[S@@](=	34310585	TTXPTDCY	121.161	-1.15	0	50	AAAA											
4	NC(=O)N[	1843030	POJWUDa	158.117	-2.18	0	50	AAAA	endogenous,world										
5	O=c1[nH]	1847417	YFQOVSG	144.086	-1.526	0	50	AAAA	biogenic,in-vitro										
6	NC(=O)C[	9256947	JWRVCVW	157.217	-1.105	0	50	AAAA	in-vitro										
7	CNC(=O)c	19844301	AFLHLPQF	141.134	-1.254	0	50	AAAA											
8	NCc1cc(=	26507110	UMEHGP)	141.13	-1.066	0	50	AAAA	in-vitro										
9	CS(=O)(=	63014624	ZQJQZDH	152.219	-1.116	0	50	AAAA											
10	NC(=O)[C	2.43E+08	TTWXSNI	154.169	-1.248	0	50	AAAA											
11	N[C@@H]	1.05E+08	SNJHTJUV	148.162	-2.586	0	50	AAAA											
12	CNS(=O)(	2.38E+08	DRWMVY	194.256	-1.476	0	50	AAAA											
13	O=c1[nH]	55860	SIVJKAW	197.194	-1.1	0	50	AAAA	in-vitro										
14	NCCOCCC	1651868	IWBOPFC	148.206	-1.063	0	50	AAAA	in-vitro										
15	CS(=O)(=	44594710	MASSUGV	187.242	-1.681	0	50	AAAA	in-vitro										
16	NS(=O)(=	1.61E+08	DRBVAWC	180.229	-1.042	0	50	AAAA											
17	CN1C(=O	1532166	MJCZWP	128.087	-1.305	0	50	AAAA	biogenic,in-vitro										
18	COC(=O)[	32098738	XWBUDP)	146.146	-1.638	0	50	AAAA	biogenic,in-vitro										
19	NCCS(=O	38251012	XZILNQFH	153.203	-1.648	0	50	AAAA											
20	O=C1N[C	2251996	PKNZKFF	186.171	-2.031	0	50	AAAA	biogenic										
21	O=C1O[C	71786849	QPIRIFWI	174.152	-1.842	0	50	AAAA											
22	O=C(O)C	1532571	WPAMZTV	178.14	-2.256	0	50	AAAA	in-vitro,metabolite										
23	CC(=O)N	35100900	GSMTYNS	174.2	-1.379	0	50	AAAA											
24	Cn1nc(CN	39064903	WMUKXU	154.173	-1.022	0	50	AAAA											
25	CNS(=O)(	1.6E+08	BLCRPDV	167.234	-1.611	0	50	AAAA											
26	O=C(O)C	3.07E+08	IWHRRKR	156.185	-1.12	0	50	AAAA											
27	NCCc1nnc	4.09E+08	PTNKDQU	179.183	-1.081	0	50	AAAA											
28	O=C(CO)[	902219	ZAQJHRI	150.13	-2.738	0	50	AAAA	biogenic,in-vivo										
29	Cn1ncc2c	16969122	HAFKXNV	180.171	-1.058	0	50	AAAA											
30	O=C1NCC	4204973	QCMGTM	184.117	-1.043	0	50	AAAA											
31	NNc1n[nH]	12958520	RDIMQHB	146.179	-1.06	0	50	AAAA	in-vitro										
32	Nc1nc(N)r	16889989	KMZCSSC	166.148	-1.734	0	50	AAAA											
33	N[C@@H]	34413647	CZCJKVCI	102.137	-1.722	0	50	AAAA											
34	COCIC@H	34426550	LNBDQMY	104.153	-1.081	0	50	AAAA	in-vitro										

준비 최근 접근: 사용할 수 없음

1 http://files.docking.org/2D/AA/AAAA.txt

2 http://files.docking.org/2D/AA/AAAB.txt

3 http://files.docking.org/2D/AA/AAAC.txt

4 http://files.docking.org/2D/AA/AAAD.txt

5 http://files.docking.org/2D/AA/AABA.txt

6 http://files.docking.org/2D/AA/AABB.txt

7 http://files.docking.org/2D/AA/AABC.txt

8 http://files.docking.org/2D/AA/AABD.txt

9 http://files.docking.org/2D/AA/AACA.txt

10 http://files.docking.org/2D/AA/AACB.txt

11 http://files.docking.org/2D/AA/AACC.txt

12 http://files.docking.org/2D/AA/AACD.txt

13 http://files.docking.org/2D/AA/AAEA.txt

14 http://files.docking.org/2D/AA/AAEB.txt

15 http://files.docking.org/2D/AA/AAEC.txt

16 http://files.docking.org/2D/AA/Aaed.txt

17 http://files.docking.org/2D/BA/BAAA.txt

18 http://files.docking.org/2D/BA/BAAB.txt

19 http://files.docking.org/2D/BA/BAAC.txt

20 http://files.docking.org/2D/BA/BAAD.txt

21 http://files.docking.org/2D/BA/BABA.txt

22 http://files.docking.org/2D/BA/BABB.txt

23 http://files.docking.org/2D/BA/BABC.txt

24 http://files.docking.org/2D/BA/BABD.txt

25 http://files.docking.org/2D/BA/BACA.txt

26 http://files.docking.org/2D/BA/BACB.txt

27 http://files.docking.org/2D/BA/BACC.txt

28 http://files.docking.org/2D/BA/BACD.txt

29 http://files.docking.org/2D/BA/BAEA.txt

30 http://files.docking.org/2D/BA/BAEB.txt

31 http://files.docking.org/2D/BA/BAEC.txt

32 http://files.docking.org/2D/BA/BAED.txt

33 http://files.docking.org/2D/CA/CAAA.txt

34 http://files.docking.org/2D/CA/CAAB.txt

35 http://files.docking.org/2D/CA/CAAC.txt

36 http://files.docking.org/2D/CA/CAAD.txt

37 http://files.docking.org/2D/CA/CABA.txt

38 http://files.docking.org/2D/CA/CABB.txt

39 http://files.docking.org/2D/CA/CABC.txt

40 http://files.docking.org/2D/CA/CABD.txt

41 http://files.docking.org/2D/CA/CACA.txt

42 http://files.docking.org/2D/CA/CACB.txt

43 http://files.docking.org/2D/CA/CACC.txt

44 http://files.docking.org/2D/CA/CACD.txt

45 http://files.docking.org/2D/CA/CAEA.txt

46 http://files.docking.org/2D/CA/CAEB.txt

47 http://files.docking.org/2D/CA/CAEC.txt

과거 빅데이터를 분석하면서 배운 교훈

- 목적: 딥러닝 모델 개발에 필요한 데이터 확보를 위함
- ZINC에 있는 물질들은 어떤 분포를 갖고 있는지 확인
 - 제품군 별로 화학구조의 분포가 다름. (의약품, 농약, 화장품, 건강기능식품 등)
- 대용량 데이터를 다루면서 당황스러웠던 점
 - 파일별로 물질 개수가 다르다. 나중에 병렬화해서 처리할 때 속도 지연 문제 발생.
 - 데이터 용량 급증 문제 (물질 → 분자지문 → 예측)
 - 파일을 이동 속도는 파일 용량 (크기)보다 개수가 더 중요하다. (용량이 적어도 개수가 너무 많으면 시간이 더 오래 걸림)
 - 운영체제에는 용량 제한만 있는 것이 아니라, 파일 개수 제한도 있음. (용량이 충분해도, 파일시스템에서 다룰 수 있는 개수를 초과하면 더 이상 저장이 불가능)

ZINC DB 분석의 목적

- 농약의 유해성을 예측하는 모델 개발 (피부 독성)
- 농약의 독성 예측 모델 개발에 사용될 수 있는 추가 데이터 확보가 목적
 - 농약 데이터는 다음 시간에 공유할 예정
- ZINC DB 물질 중에서 농약과 유사한 구조는 몇 개나 있을까?
 - 전체 다 비교한다면.... 8억 X 1000개 = 8000억 번의 계산이 필요
- 8억개 전부를 다루기 전에 소수의 샘플데이터에 코드 테스트
 - AAAA.txt 파일에서 간단한 분석을 진행
 - 추후에 작성한 코드를 8억개 전체에 적용

농약
(~1000개)

ZINC DB
(~8억)

AAAA.txt에 대해서 진행해보고 싶은 분석

- 파일에 있는 물질들의 분포 분석
 - AAAA.txt 안에 들어있는 물질 개수는 총 몇 개?
 - MW (molecular weight): 분자의 크기 (분자를 구성하고 있는 원자 무게의 총합)
 - logP (partition coefficient): 분자의 지용성 (기름에 잘 녹는지를 확인)
 - 물질 데이터의 분포를 확인할 때, 많이 체크하는 성질.
 - MW, logP의 범위는? (최대값, 최소값 찾기)
 - logP < 5 인 물질은 총 몇 개? (ZINC DB 전체에서 찾아보기)
 - MW, logP의 분포를 scatter plot으로 확인해보기
- 분석의 목적: 현재 보유하고 있는 데이터에는 얼마나 다양한 구조가 포함되어 있는지 확인하기 위함.
 - 수십만개를 모두 직접 확인하는 것은 어려움. MW, logP 값의 다양성으로 구조 다양성을 대신 확인.

Python을 사용해서 분석 해야 하는 이유?

- 데이터 분석을 파일 별로 여러 차례 반복해야 함.
 - ZINC-downloader-2D-txt.uri 파일에는 총 1916개의 파일
 - Pandas로 분석 과정을 자동화 해야 각 파일에 동일한 분석을 적용할 수 있음
- 데이터 분석 편의성 (친해지려면 시간이 조금 필요하지만...)
 - 데이터 개수 확인 (shape)
 - 최대값, 최소값 (max, min)
 - 특정 조건에 맞는 물질만 찾아내는 방법을 subset한다고 표현합니다. Subset 방법
 - 데이터 분포 시각화 (matplotlib): scatter plot (x 축: mw, y축: logP)

Input/output

pandas.read_pickle

pandas.DataFrame.to_pickle

pandas.read_table

pandas.read_csv

pandas.DataFrame.to_csv

pandas.read_fwf

pandas.read_clipboard

pandas.DataFrame.to_clipboard

pandas.read_excel

pandas.DataFrame.to_excel

pandas.ExcelFile

pandas.ExcelFile.book

pandas.ExcelFile.sheet_names

pandas.ExcelFile.parse

pandas.io.formats.style.Styler.to_excel

pandas.ExcelWriter

pandas.read_json

pandas.json_normalize

pandas.DataFrame.to_json

pandas.io.json.build_table_schema

pandas.read_html

pandas.DataFrame.to_html

pandas.io.formats.style.Styler.to_html

pandas.read_xml

pandas.DataFrame.to_xml

pandas.read_csv

```
pandas.read_csv(filepath_or_buffer, *, sep=<no_default>, delimiter=None,
header='infer', names=<no_default>, index_col=None, usecols=None, dtype=None,
engine=None, converters=None, true_values=None, false_values=None,
skipinitialspace=False, skiprows=None, skipfooter=0, nrows=None, na_values=None,
keep_default_na=True, na_filter=True, skip_blank_lines=True, parse_dates=None,
date_format=None, dayfirst=False, cache_dates=True, iterator=False,
chunksize=None, compression='infer', thousands=None, decimal='.',
lineterminator=None, quotechar='"', quoting=0, doublequote=True, escapechar=None,
comment=None, encoding=None, encoding_errors='strict', dialect=None,
on_bad_lines='error', low_memory=True, memory_map=False, float_precision=None,
storage_options=None, dtype_backend=<no_default>) #
```

[\[source\]](#)

Read a comma-separated values (csv) file into DataFrame.

Also supports optionally iterating or breaking of the file into chunks.

Additional help can be found in the online docs for [IO Tools](#).

Parameters:

filepath_or_buffer : str, path object or file-like object

Any valid string path is acceptable. The string could be a URL. Valid URL schemes include http, ftp, s3, gs, and file. For file URLs, a host is expected. A local file could be: <file://localhost/path/to/table.csv>.

If you want to pass in a path object, pandas accepts any `os.PathLike`.

By file-like object, we refer to objects with a `read()` method, such as a file handle (e.g. via builtin `open` function) or `StringIO`.

sep : str, default ','

Character or regex pattern to treat as the delimiter. If `sep=None`, the C engine cannot automatically detect the separator, but the Python parsing engine can, meaning the latter will be used and automatically detect the separator from only the first valid row

On this page

[read_csv\(\)](#)



Input/output

- pandas.read_pickle
- pandas.DataFrame.to_pickle
- pandas.read_table
- pandas.read_csv**
- pandas.DataFrame.to_csv
- pandas.read_fwf
- pandas.read_clipboard
- pandas.DataFrame.to_clipboard
- pandas.read_excel
- pandas.DataFrame.to_excel
- pandas.ExcelFile
- pandas.ExcelFile.book
- pandas.ExcelFile.sheet_names
- pandas.ExcelFile.parse
- pandas.io.formats.style.Styler.to_excel
- pandas.ExcelWriter
- pandas.read_json
- pandas.json_normalize
- pandas.DataFrame.to_json
- pandas.io.json.build_table_schema
- pandas.read_html
- pandas.DataFrame.to_html
- pandas.io.formats.style.Styler.to_html
- pandas.read_xml

Examples

```
>>> pd.read_csv("data.csv")
   Name  Value
0   foo      1
1   bar      2
2  #baz      3
```

Index and header can be specified via the *index_col* and *header* arguments.

```
>>> pd.read_csv("data.csv", header=None)
   0      1
0  Name  Value
1   foo      1
2   bar      2
3  #baz      3
```

```
>>> pd.read_csv("data.csv", index_col="Value")
   Name
Value
1    foo
2    bar
3  #baz
```

Column types are inferred but can be explicitly specified using the *dtype* argument.

```
>>> pd.read_csv("data.csv", dtype={"Value": float})
   Name  Value
0   foo    1.0
1   bar    2.0
2  #baz    3.0
```

True, False, and NA values, and [other dtypes](#) have defaults, but can be explicitly specified, too. Supply the values you would like as strings or lists of strings!

↑ Back to top

☰ On this page

read_csv()

Untitled notebook

출처



+ 소스 추가

웹에서 새 소스를 검색하세요



Fast Research



저장된 소스가 여기에 표시됩니다

위의 소스 추가를 클릭하여 PDF, 웹사이트, 텍스트, 동영상 또는 오디오 파일을 추가하세요. 또는 Google Drive에서 파일을 직접 가져올 수 있습니다.

채팅



사용자의 문서를 활용해 AI 오디오 및 동영상 오버뷰 만들기

웹에서 새 소스를 검색하세요



Fast Research



또는 파일 드롭

PDF, 이미지, 문서, 오디오 등

파일 업로드

웹사이트

Drive

복사된 텍스트

시작하려면 출처를 업로드하세요.

소스 0개



스튜디오



AI 오디오 오버뷰



슬라이드 자료



동영상 개요



마인드맵



보고서



플래시카드



퀴즈



인포그래픽



데이터 표



스튜디오 출력이 여기에 저장됩니다

소스를 추가한 후 클릭하여 AI 오디오 오버뷰, 학습 가이드, 마인드맵 등을 추가해 보세요.

메모 추가

출처

+ 소스 추가

웹에서 새 소스를 검색하세요

모든 소스 선택

pandas.read_excel — pandas 3.0.1 documenta...

프롬프트 작성 시 주의사항

개인의 프로그래밍 경험을 언급.

(python 처음. 배우긴 했는데 자신

없음. 등) 프로그래밍 실력에 따라

AI 답변의 유용성이 결정됨.

채팅

txt 파일을 pandas로 열려면 어떻게 해야되지?

텍스트(.txt) 파일을 pandas로 읽어올 때는 파일 내부의 데이터가 어떻게 구분되어 있는지에 따라 `read_csv` 또는 `read_table` 함수를 사용합니다 1 2.

엑셀 파일을 읽을 때와 마찬가지로 매우 간단하게 작성할 수 있습니다.

1. 기본 코드 예시

가장 많이 사용되는 방식은 `read_csv` 함수를 사용하는 것입니다. 소스 문헌의 'See also' 항목을 보면 CSV 파일을 읽을 때 이 함수를 사용한다고 명시되어 있습니다 2.

```
import pandas as pd
```

```
# 1. 쉼표(,)로 구분된 txt 파일인 경우
df = pd.read_csv('data.txt')
```

```
# 2. 탭(Tab)으로 구분된 txt 파일인 경우
df = pd.read_table('data.txt')
```

```
# 데이터 확인
df.head()
```

2. 상황별 함수 선택 (소스 문헌 기준)

소스 문헌에서는 데이터의 형식에 따라 다음과 같은 함수들을 제공하고 있습니다 1:

- `read_csv`: 쉼표(,)로 구분된 텍스트 파일을 읽을 때 주로 사용합니다 2.
- `read_table`: 일반적인 표 형식의 텍스트 파일을 읽을 때 사용하며, 기본적으로 탭을 구분자로 인식하는 경우가 많습니다 1.
- `read_fwf`: 데이터가 일정한 너비(Fixed-width)로 정렬된 경우에 사용합니다 2.

입력을 시작하세요...

소스 1개

스튜디오

AI 오디오 오버뷰

슬라이드 자료

동영상 개요

마인드맵

보고서

플래시카드

퀴즈

인포그래픽

데이터 표

스튜디오 출력이 여기에 저장됩니다

소스를 추가한 후 클릭하여 AI 오디오 오버뷰, 학습 가이드, 마인드맵 등을 추가해 보세요.

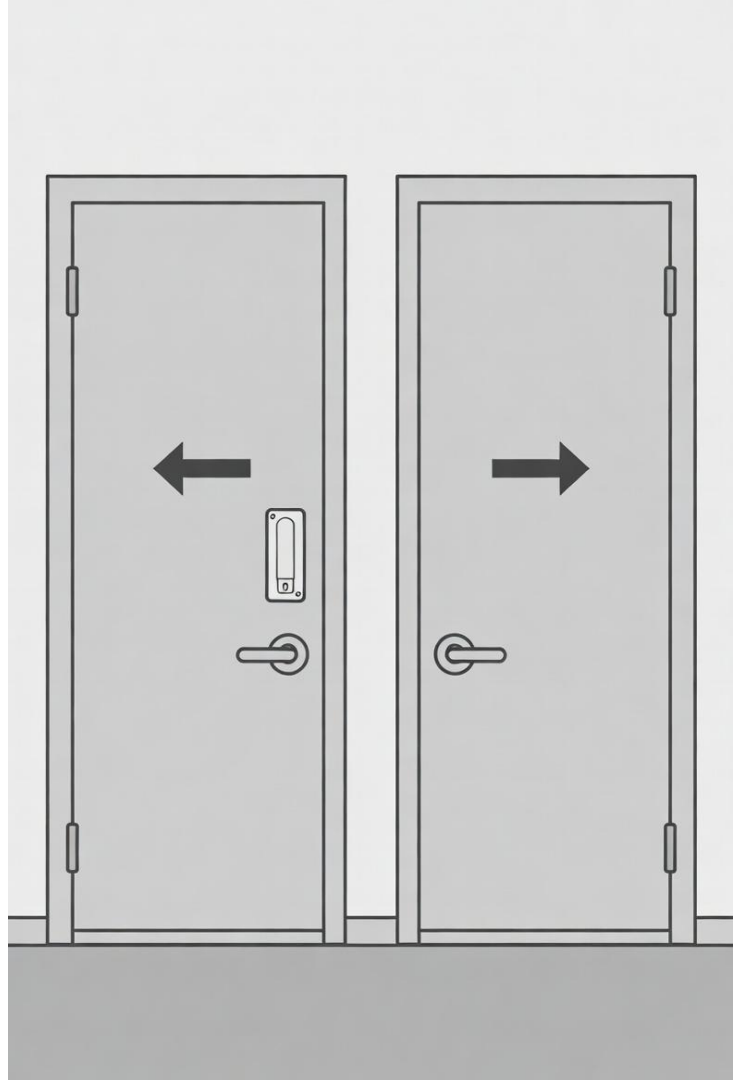
메모 추가

Jupyter notebook

1. txt 파일 열기 (테이블 형태이므로 pandas 사용)
2. 오류 메시지 읽는 방법 & 필요 라이브러리 설치 방법
3. Pandas에서 엑셀 필터와 같은 기능 사용하는 법 (subset)
4. Pandas shape 사용 (데이터 개수 파악)
5. Max, min을 이용해서 데이터 확인 방법
6. Matplotlib을 이용한 scatterplot 그리기

아마존의 의사결정 방식

- One-way door vs two-way door
 - Two-way door: 실수해도 문제 없음. 언제든지 되돌아올 수 있으면, 크게 고민하지 말고 실행.
 - One-way door: 신중하게 선택. 다시 돌아오기 어렵다면, 좀 더 신중한 의사결정.
- 수업에서 진행되는 실습은 모두 two-way door.
 - 데이터 처리 실수하면? (다시 다운 받으면 됨)
 - 만약에 컴퓨터에 문제가 생기면? (수리하면 됨)



수업 관련 유의 사항

- 피드백은 짧은 간격으로 자주 해주세요. (소통 과정에서의 오류 예방)
- 수업을 진행하고 있는 사람은 잘 인지하지 못하는 오류.
 - 실습이 잘 진행되는지 확인하다 보면, 정신이 없음. 잘못된 내용 혹은 더 나은 방법이 있다면 꼭 제안할 것.
- 수업 결과물은 개인이 학기 말에 1회 제출하는 보고서입니다.
 - 초반에 조금 뒤쳐져도 괜찮습니다. 주변 학생들과 저에게 자주 도움 요청하시면서 수업 따라가면 보고서 제출에는 문제가 없는 수준으로 성장할 겁니다.
- 문제가 잘 해결이 안되면, 일어서서 돌아다니는 것도 도움이 됩니다.
 - 사람의 사고는 사용 가능한 공간에 상당히 영향을 받음.
 - 도움 주는 사람도 이동할 수 있지만, 도움이 필요한 사람도 이동해서 도움 요청.

데이터 현황 파악을 위한 분석

- Python에서 파일 열기
 - ZINC-downloader-2D-txt.uri 파일은 어떻게 파이썬에서 열어야 할까?
- 개수 확인하기
 - 다운로드를 모두 받았는지 검증할 때, 웹페이지 상에 있는 숫자와 직접 계산한 개수 비교
 - AAAA.txt 파일에는 총 몇 개의 물질이 있을까?
 - 다운받을 수 있는 txt 파일들은 총 몇 개?
 - Python 자료구조 활용 방법: list, dictionary
(ZINC DB를 다운받고, 전체 데이터 현황 정리에 필요)